SQL ASSIGNMENT 11

Subject: Database Management Systems (SQL) **Topic:** Common Table Expressions (CTEs & Recursive CTEs) **Status:** Completed

QUESTION 1

Create a CTE using the `WITH` clause to select all products with a rating above 4.5 from a `Products` table (Flipkart/Myntra top-rated items).

```
WITH TopRatedProducts AS ( SELECT product_id, product_name, category, rating, price FROM Products WHERE rating > 4.5 ) SELECT * FROM TopRatedProducts ORDER BY rating DESC;
```

QUESTION 2

Rewrite a query to find all restaurants in 'Ahmedabad' with delivery charges under 50 using a subquery and a CTE, then compare readability.

```
-- Subquery Approach SELECT * FROM ( SELECT id, name, city, delivery_charge FROM Restaurants WHERE city = 'Ahmedabad' ) AS AhmedabadBranch WHERE delivery_charge < 50; -- CTE Approach WITH LowDeliveryAhmedabad AS ( SELECT id, name, city, delivery_charge FROM Restaurants WHERE city = 'Ahmedabad' AND delivery_charge < 50 ) SELECT * FROM LowDeliveryAhmedabad;
```

Readability Comparison: The CTE version reads sequentially top-to-bottom, separating logic definition from final execution, whereas the subquery forces inside-out reading and requires aliasing outer wrappers.

QUESTION 3

Using two CTEs in a single query, find top 3 most-followed users and top 3 most-liked posts, returning both lists in one result set (Instagram-style).

```
WITH TopUsers AS ( SELECT 'Top User' AS entity_type, username AS name_or_title, followers_count AS metric FROM Users ORDER BY followers_count DESC LIMIT 3 ), TopPosts AS ( SELECT 'Top Post' AS entity_type, caption AS name_or_title, likes_count AS metric FROM Posts ORDER BY likes_count DESC LIMIT 3 ) SELECT * FROM TopUsers UNION ALL SELECT * FROM TopPosts;
```

QUESTION 4

Write a recursive CTE generating dates for the next 7 days starting from today (BookMyShow booking slots).

```
WITH RECURSIVE UpcomingDates AS ( -- Anchor member: Today's date SELECT CURRENT_DATE AS booking_date, 1 AS day_count UNION ALL -- Recursive member: Add 1 day until reaching 7 days SELECT (booking_date + INTERVAL '1 day')::DATE, day_count + 1 FROM UpcomingDates WHERE day_count < 7 ) SELECT booking_date FROM UpcomingDates;
```

QUESTION 5

Refactor a messy unreadable query finding users with > 1000 followers into a clean CTE structure.

```
-- Clean Refactored CTE Query WITH HighFollowerUsers AS ( SELECT id, username, email, followers_count, created_at FROM Users WHERE followers_count > 1000 ) SELECT id, username, followers_count FROM HighFollowerUsers ORDER BY followers_count DESC;
```